

Assignment1

March 7, 2025

```
[3]: '''
Names: Prashant Ram, Muhammad Hassan, Mohamed Burhan, Gandhar Tare
Course: CSC 177
Assignment: 1 - Data Preprocessing
Professor: Jagannadha Chidella
Details: This project deals with preprocessing data from a .csv file to ready_
        it for
        data modelling.
'''

import pandas
import matplotlib.pyplot as matPlot
import seaborn
```

```
[4]: '''
This block reads the data from a .csv file and makes columns for the data
'''
try:
    # import the data from the .csv file. "data" = dataframe
    data = pandas.read_csv('lung_disease_data.csv')

    #print the data attribute columns
    print(data.columns)

    # print the number of rows and columns
    print(f'We have {data.shape[0]} rows')
    print(f'We have {data.shape[1]} columns')

    # print the first 5 rows of the data
    print('\n', data.head())

except Exception as e:
    print("An error occured while trying to read from the CSV file", e)
```

```
Index(['Age', 'Gender', 'Smoking Status', 'Lung Capacity', 'Disease Type',
       'Treatment Type', 'Hospital Visits', 'Recovered'],
      dtype='object')
We have 5200 rows
```

We have 8 columns

	Age	Gender	Smoking Status	Lung Capacity	Disease Type	Treatment Type	\
0	71.0	Female	No	4.49	COPD	Therapy	
1	34.0	Female	Yes	NaN	Bronchitis	Surgery	
2	80.0	Male	Yes	1.95	COPD	NaN	
3	40.0	Female	Yes	NaN	Bronchitis	Medication	
4	43.0	Male	Yes	4.60	COPD	Surgery	

	Hospital Visits	Recovered
0	14.0	Yes
1	7.0	No
2	4.0	Yes
3	1.0	No
4	NaN	Yes

```
[7]: '''
      This block counts the number of missing values in each column
      '''

      #list to hold the columns with missing values
      missingColumns = []

      try:
          #loop through the columns and get the number of missing values
          for i in data.columns:
              #get the total of column of missing values
              total = data[i].isnull().sum()
              #display the column and the missing values number
              print(f"{i}: {total}")

              #add missing value columns to the list
              if total > 0:
                  missingColumns.append(i)

              #finally display which columns had missing values
              print(f"\nThese columns had missing data: {missingColumns}")

      #catch any reading errors
      except Exception as e:
          print("umm. An error occurred while trying to count the missing values..", e)
```

Age: 300
Gender: 300
Smoking Status: 300
Lung Capacity: 300
Disease Type: 300

Treatment Type: 300
Hospital Visits: 300
Recovered: 300

These columns had missing data: ['Age', 'Gender', 'Smoking Status', 'Lung Capacity', 'Disease Type', 'Treatment Type', 'Hospital Visits', 'Recovered']

```
[9]: '''
      This block drops all rows and columns with missing data
      '''
      # print the number of rows and columns
      original_rows = data.shape[0]
      original_columns = data.shape[1]
      print(f'We have {original_rows} [original] rows and {original_columns} columns')
      print('\nOriginal Data:', data.head())

      #drops all rows with no data
      data = data.dropna()

      clean_rows = data.shape[0]
      clean_columns = data.shape[1]
      print(f'\nWe have {clean_rows} [Clean] rows and {clean_columns} columns')
      print('\nCleaned data Data:', data.head())

      print(f'\n\nWe had {original_rows - clean_rows} rows and {original_columns -
      ↪clean_columns} columns of missing data')
```

We have 5200 [original] rows and 8 columns

Original Data:	Age	Gender	Smoking Status	Lung Capacity	Disease Type	Treatment Type \
0	71.0	Female	No	4.49	COPD	Therapy
1	34.0	Female	Yes	NaN	Bronchitis	Surgery
2	80.0	Male	Yes	1.95	COPD	NaN
3	40.0	Female	Yes	NaN	Bronchitis	Medication
4	43.0	Male	Yes	4.60	COPD	Surgery

	Hospital Visits	Recovered
0	14.0	Yes
1	7.0	No
2	4.0	Yes
3	1.0	No
4	NaN	Yes

We have 3236 [Clean] rows and 8 columns

Cleaned data Data:	Age	Gender	Smoking Status	Lung Capacity	Disease Type	Treatment Type \
--------------------	-----	--------	----------------	---------------	--------------	------------------

0	71.0	Female	No	4.49	COPD	Therapy
5	22.0	Female	No	3.65	Bronchitis	Medication
6	41.0	Male	Yes	2.98	Asthma	Medication
8	21.0	Female	No	4.05	Lung Cancer	Medication
9	49.0	Male	Yes	5.70	Pneumonia	Medication

	Hospital Visits	Recovered
0	14.0	Yes
5	11.0	Yes
6	12.0	Yes
8	12.0	No
9	11.0	Yes

We had 1964 rows and 0 columns of missing data

```
[11]: '''
This block converts the categorical columns to numeric types so they can be
    ↪compared
'''
#Convert gender column to numeric
data['Gender'] = data['Gender'].map({'Male': 0, 'Female': 1})
print('Gender Column.....DONE')

#Convert Smoking Status Column to Numeric
data['Smoking Status'] = data['Smoking Status'].map({'Yes': 1, 'No': 0})
print('Smoking Status Column.....DONE')

#Convert Recovered Column to Numeric
data['Recovered'] = data['Recovered'].map({'Yes': 1, 'No': 0})
print('Recovered Column.....DONE')

#Convert Disease and Treatment Type Column to Numeric
data = pandas.get_dummies(data, columns=['Disease Type', 'Treatment Type'],
    ↪drop_first=True)
print('Disease Column.....DONE')
print('Treatment Column.....DONE')
```

```
Gender Column...DONE
Smoking Status Column...DONE
Recovered Column...DONE
Disease Column...DONE
Treatment Column...DONE
```

```
[13]: # Convert boolean columns to integers 1 for True and 0 for False
data = data.astype(int)
```

```
#save the dataset using the new split data columns for Disease Type and
↳Treatment Type
try:
    data.to_csv('lung_disease_cleaned.csv', index=False)
    print('Data saved to <<<lung_disease_cleaned.csv>>> with new columns
↳WITHOUT converting TRUE/FALSE values to binary!')
    print('\n', data.head())
except Exception as e:
    print("Couldn't save the cleaned data file!")
```

Data saved to <<<lung_disease_cleaned.csv>>> with new columns WITHOUT converting TRUE/FALSE values to binary!

	Age	Gender	Smoking Status	Lung Capacity	Hospital Visits	Recovered	\
0	71	1	0	4	14	1	
5	22	1	0	3	11	1	
6	41	0	1	2	12	1	
8	21	1	0	4	12	0	
9	49	0	1	5	11	1	

	Disease Type_Bronchitis	Disease Type_COPD	Disease Type_Lung Cancer	\
0	0	1	0	
5	1	0	0	
6	0	0	0	
8	0	0	1	
9	0	0	0	

	Disease Type_Pneumonia	Treatment Type_Surgery	Treatment Type_Therapy
0	0	0	1
5	0	0	0
6	0	0	0
8	0	0	0
9	1	0	0

```
[15]: '''
Plots the data from the current dataframe for the individual columns
'''
#plot the Gender column
matPlot.subplot(2,2,2)
seaborn.countplot(x='Gender', data=data, hue='Gender', palette='viridis',
↳legend=False)
matPlot.title('Gender of Patient')

#plot the Smoking Status column
matPlot.subplot(2,2,3)
seaborn.countplot(x='Smoking Status', data=data, hue='Smoking Status',
↳palette='viridis', legend=False)
```

```

matPlot.title('Smoking Status of Patient')

#plot the Recovered column
matPlot.subplot(2,2,4)
seaborn.countplot(x='Recovered', data=data, hue='Recovered', palette='viridis',
    ↪legend=False)
matPlot.title('Did the Patient Recover?')

#make the charts fit nicely together in one window
matPlot.tight_layout()

'''
Histogram plot of numerical columns
'''
# Plot histograms for numerical columns
matPlot.figure(figsize=(15, 5))

# Age
matPlot.subplot(1, 3, 1)
seaborn.histplot(data['Age'], kde=True, color='red')
matPlot.title('Histogram of Age of Patients')

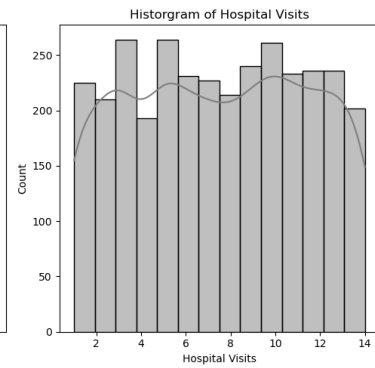
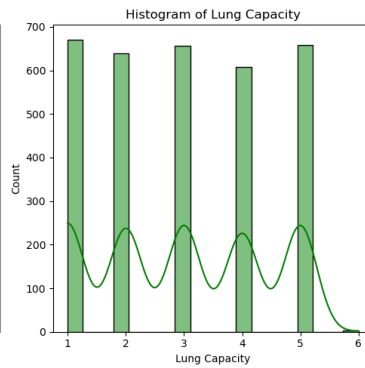
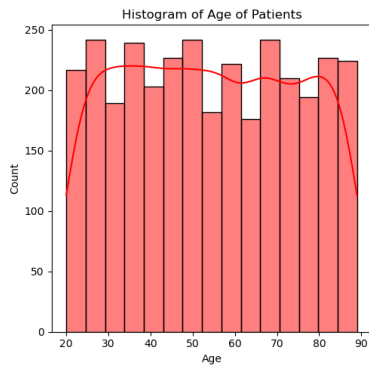
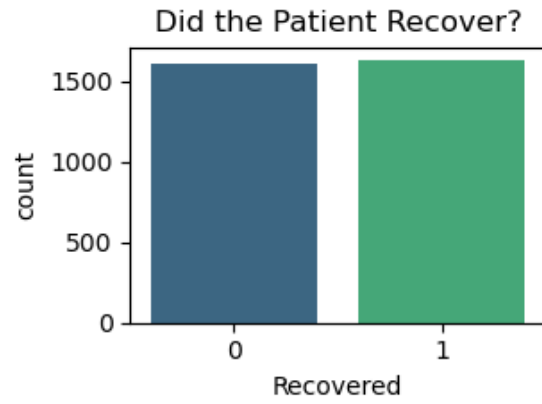
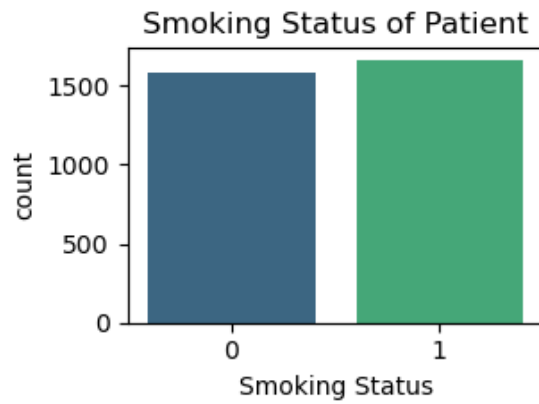
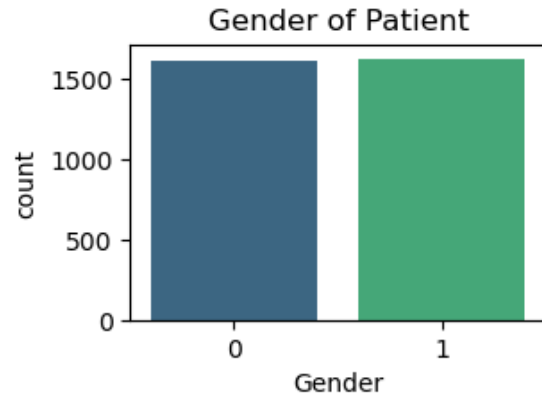
# Lung Capacity
matPlot.subplot(1, 3, 2)
seaborn.histplot(data['Lung Capacity'], kde=True, color='green')
matPlot.title('Histogram of Lung Capacity')

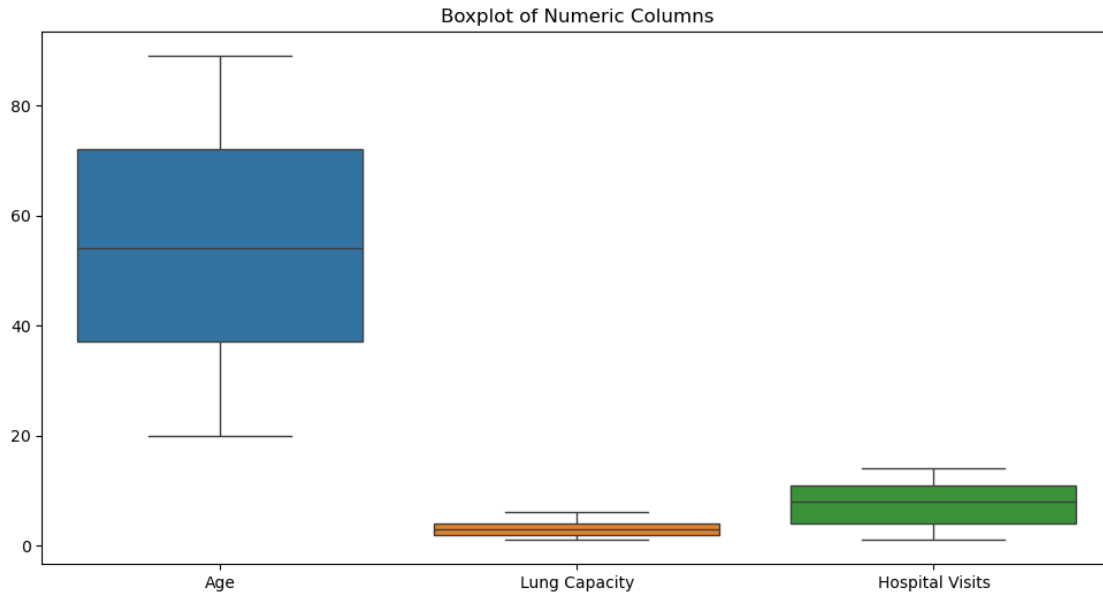
# Hospital Visits
matPlot.subplot(1, 3, 3)
seaborn.histplot(data['Hospital Visits'], kde=True, color='gray')
matPlot.title('Histogram of Hospital Visits')

matPlot.tight_layout()
matPlot.show()

'''
Boxplot of numerical columns
'''
#choose figure size for the plot area
matPlot.figure(figsize=(12, 6))
seaborn.boxplot(data=data[['Age', 'Lung Capacity', 'Hospital Visits']])
matPlot.title("Boxplot of Numeric Columns")
#display the data
matPlot.show()

```





```
[17]: '''
This block plots the Disease columns with the Age column.
'''

#melt the disease type columns into a single column
diseaseTypes = ['Disease Type_Bronchitis', 'Disease Type_COPD', 'Disease_
↳Type_Lung Cancer', 'Disease Type_Pneumonia']
data_melted = data.melt(id_vars=['Age'], value_vars=diseaseTypes,
↳var_name='Disease Type', value_name='Has Disease')

#only get the rows with the disease
data_melted = data_melted[data_melted['Has Disease'] == 1]

#plot the data
matplotlib.figure(figsize=(10, 6))
seaborn.boxplot(x='Disease Type', y='Age', data=data_melted)
matplotlib.title('Age Distribution by Disease Type')
matplotlib.xlabel('Disease Type')
matplotlib.ylabel('Age')
matplotlib.show()

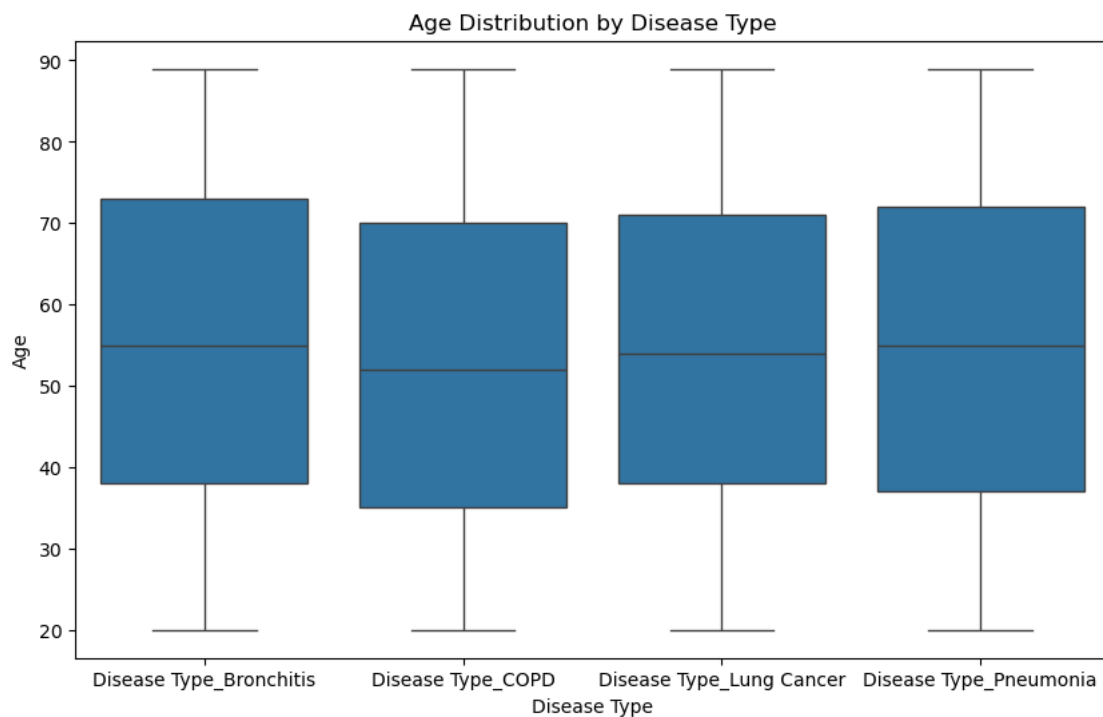
#Interpretation
'''
It seems that patients suffering from Bronchitis had 1/4 of them of ages
↳between 20 - 38 and the other 1/4 between 74 - 89. The median age
group (1/2) suffering from this diseases were about 55.
For COPD, the first 1/4 were between 20 -35 and their upper 1/4 were between 70_
↳- 89 while the median age was about 53.
```



```

For Lung cancer, the first 1/4 were between 20 -35 and their upper 1/4 were
↳between 70 - 89 while the median age was about 54.
For Pnuemonia, the first 1/4 were between 20 -38 and their upper 1/4 were
↳between 72 - 89 while the median age was about 55.
'''
#Insights
'''
Based on the interpretation above, we can predict that people who suffer from
↳lung diseases majority are in their mid 50's with the older age group
(greater than 70) suffering from Bronchitis and Pneumonia.
'''

```



```

[17]: "\nBased on the interpretation above, we can predict that people who suffer from
lung diseases majority are in their mid 50's with the older age group\n(greater
than 70) suffering from Bronchitis and Pneumonia.\n"

```

```

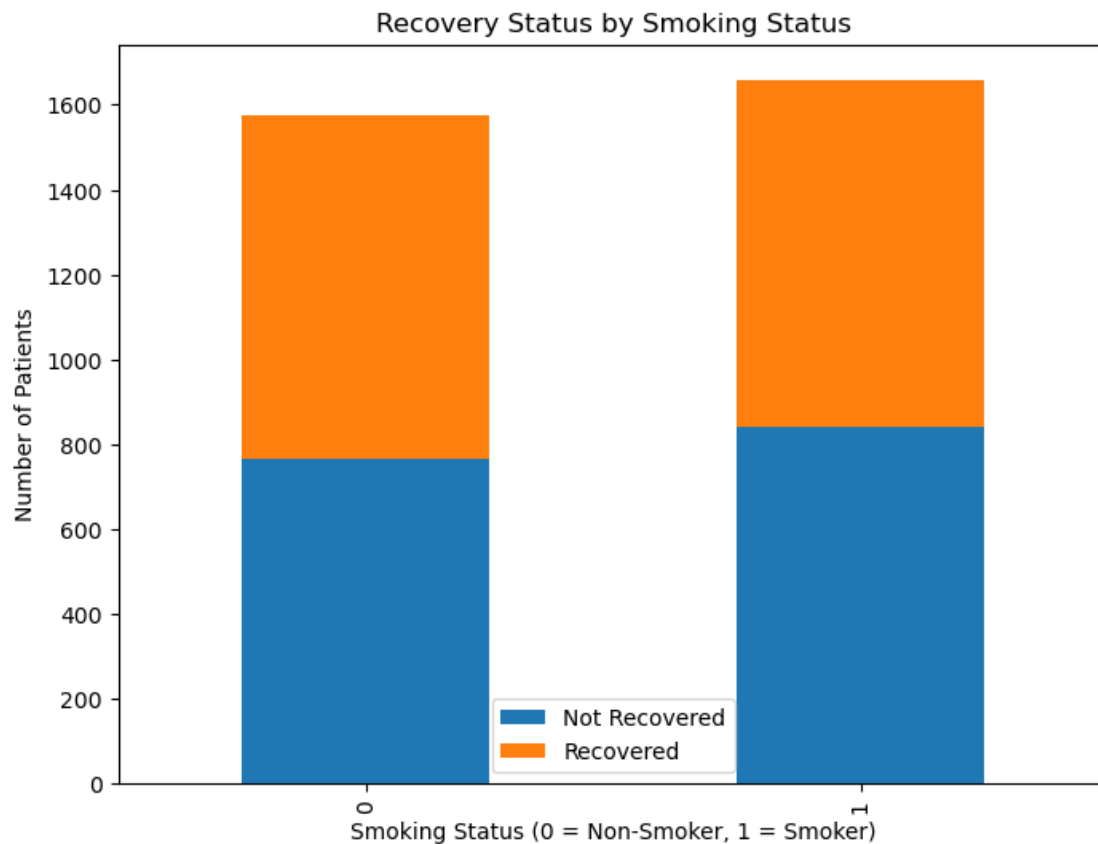
[19]: '''
This block plots the Smoking Status column with the Recovered column.
'''
#group by smoking status and recovery
smoking_recovery = data.groupby(['Smoking Status', 'Recovered']).size().
↳unstack()

```

```
#plot the data
smoking_recovery.plot(kind='bar', stacked=True, figsize=(8, 6))
matPlot.title('Recovery Status by Smoking Status')
matPlot.xlabel('Smoking Status (0 = Non-Smoker, 1 = Smoker)')
matPlot.ylabel('Number of Patients')
matPlot.legend(['Not Recovered', 'Recovered'])
matPlot.show()

#Intepretation
'''
The stacked bar graph depicts that patients who were Non-smokers had about
↳similar recovery outcomes as the ones
who smoked.
However, we can clearly see that Smokers had a higher outcome of not recovering
↳from their lung diseases.
'''

#Insight
'''
The insight that we can make from this is that smoking can worsen the recovery
↳outcome of lung diseases.
'''
```



```
[19]: '\n\nThe insight that we can make from this is that smoking can worsen the recovery outcome of lung diseases.\n'
```

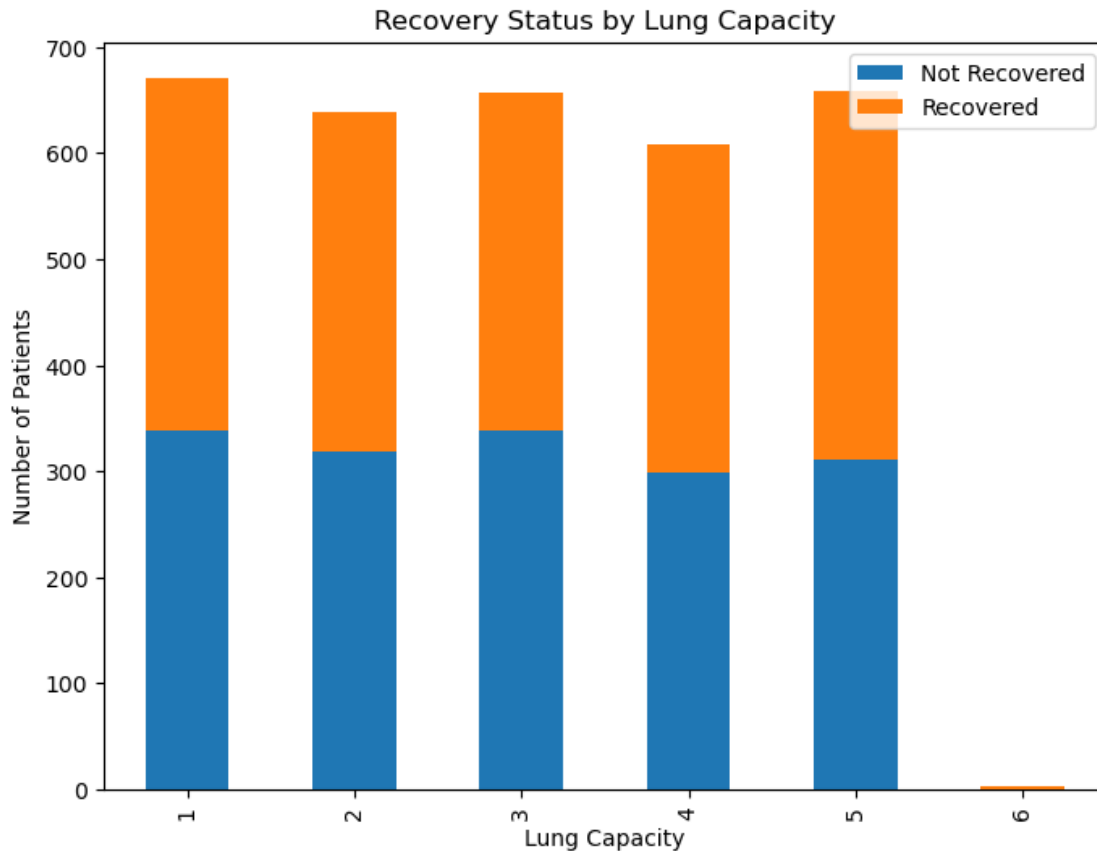
```
[21]: '''
This block plots the Lung Capacity column with the Recovered column.
'''

#group the data by the two columns
lung_capacity_recovery = data.groupby(['Lung Capacity', 'Recovered']).size().
    ↪unstack()

#plot the data
lung_capacity_recovery.plot(kind='bar', stacked=True, figsize=(8, 6))
matPlot.title('Recovery Status by Lung Capacity')
matPlot.xlabel('Lung Capacity')
matPlot.ylabel('Number of Patients')
matPlot.legend(['Not Recovered', 'Recovered'])
matPlot.show()

#Intepretation
'''
The stacked bar graph depicts that patients who had a lower lung capacity (1, 2,
    ↪and 3) had higher "Not recovered" count.
Moreover, patients with a high lung capacity (4-5) had about similar ratio of
    ↪Recovered and Not Recovered. Finally, patients with the highest
lung capacity of 6 had always recovered.
'''

#Insight
'''
The insight that we can make from this is that a lower lung capacity can
    ↪significantly reduce the recovery outcomes whereas a a very one can
almost always guarantee recovery.
'''
```



[21]: '\n\nThe insight that we can make from this is that a lower lung capacity can significantly reduce the recovery outcomes whereas a a very one can \n\nalmost always guarantee recovery.\n'

```
[23]: '''
This block plots the Hospital Visits column with the Recovered column.
'''

#make a boxplot for the two columns.
matPlot.figure(figsize=(10, 6))
seaborn.boxplot(x='Recovered', y='Hospital Visits', data=data)
matPlot.title('Hospital Visits by Recovery Status')
matPlot.xlabel('Recovery Status (0 = Not Recovered, 1 = Recovered)')
matPlot.ylabel('Hospital Visits')
matPlot.show()

#Intepretation
'''
We can see that 1/4 of the patients who had not recovered had hospital visits_
↳ between 2-4 and the upper 1/4 we between 11-14 visits.
The majority of the patients had about 8 visits.
```

In contrast, patients who did recover had a median visits of about 7.

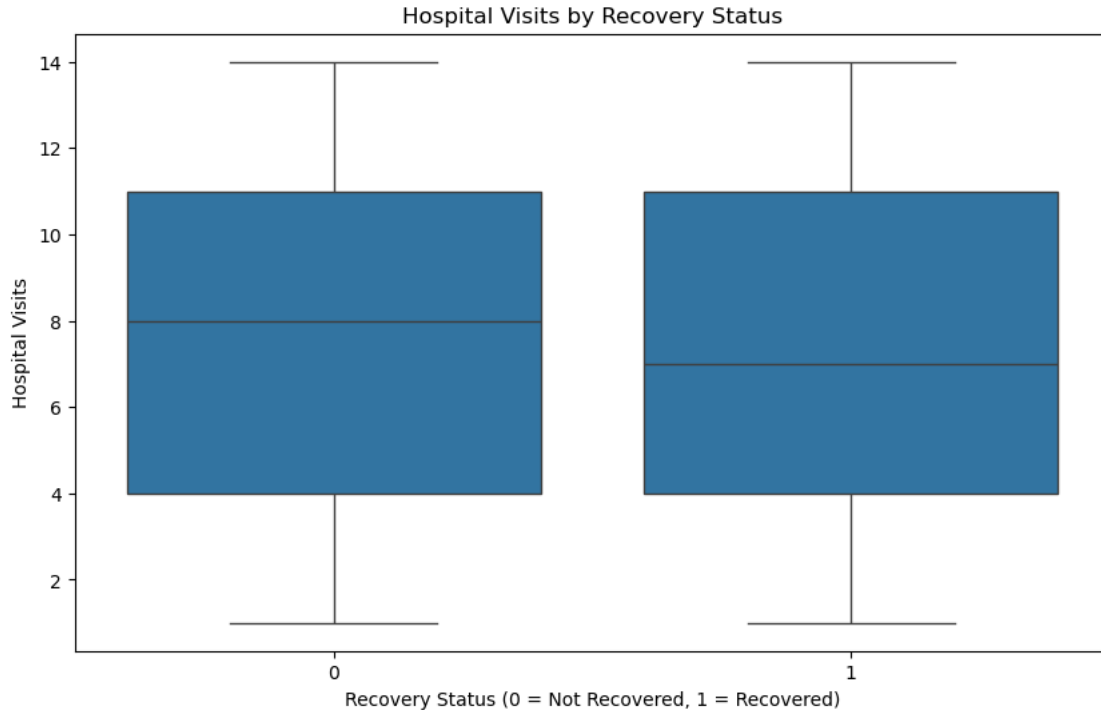
'''

#Insight

'''

*We can infer from the interpretation above that patients who have a hard time
→treating their conditions would require more hospital visits.*

'''



[23]: '\nWe can infer from the interpretation above that patients who have a hard time
treating their conditions would require more hospital visits.\n'

```
[25]: '''
This block plots the Treatment Types column with the Recovered column.
'''

#group the data by Treatment Type and Recovery
treatment_recovery = data.groupby(['Treatment Type_Surgery', 'Recovered']).
    →size().unstack()

#plot the data
treatment_recovery.plot(kind='bar', stacked=True, figsize=(8, 6))
matPlot.title('Recovery Status by Treatment Type')
matPlot.xlabel('Treatment Type (0 = Therapy, 1 = Surgery)')
```

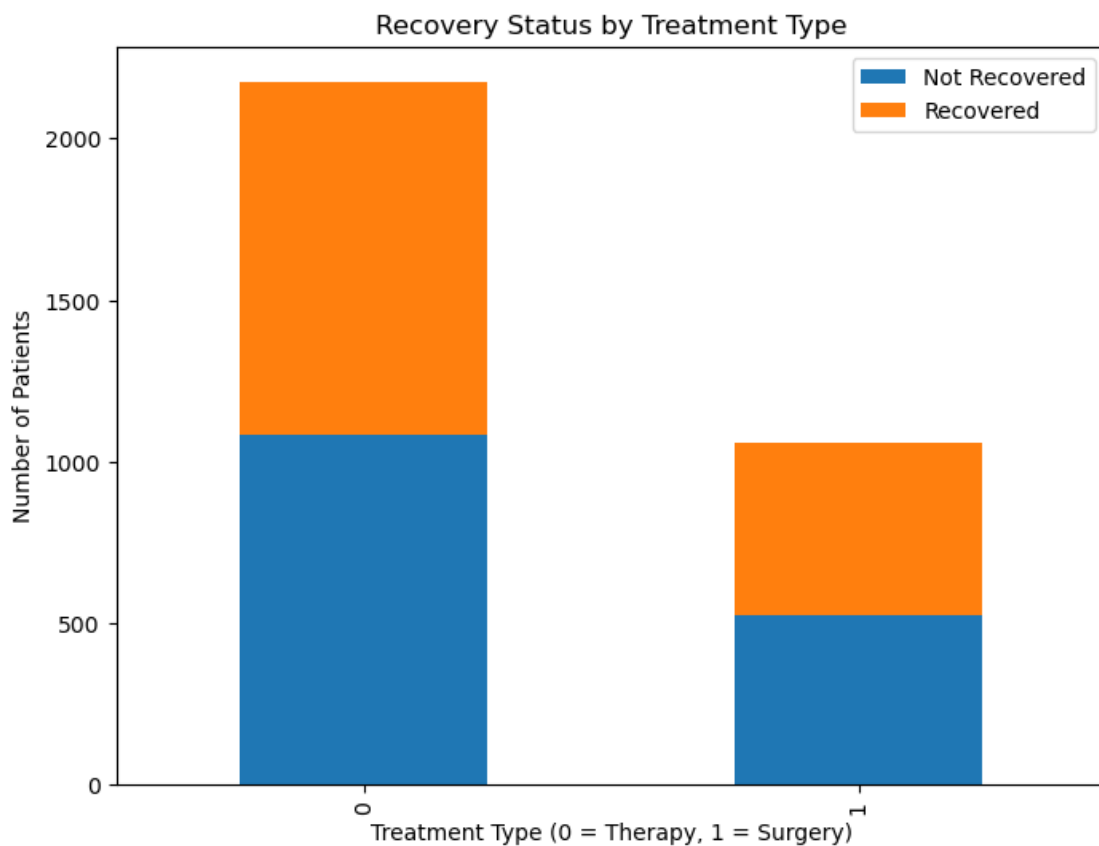
```

matPlot.ylabel('Number of Patients')
matPlot.legend(['Not Recovered', 'Recovered'])
matPlot.show()

#Interpretation
'''
We can see that patients which opted for therapy as their form of treatment had
    ↳ a higher count of not recovering when compared to the
ones who chose surgery. The ones who chose surgery had a higher count of
    ↳ recovering after the treatment.
'''

#Insight
'''
We can say that surgical forms of treatment work much better than just
    ↳ therapeutic kind.
'''

```



[25]: '\nWe can say that surgical forms of treatment work much better than just therapeutic kind.\n'

```
[27]: '''
This block splits the cleaned dataset into training and testing data.
'''

from sklearn.model_selection import train_test_split

#display what should be the expected data modelling pre-split into training and
↳testing data sets.
total_rows = data.shape[0]
training_data = int(0.8 * total_rows)
print(f'We have a total of {total_rows} rows of data.')
print(f'80% of {total_rows} is {training_data} which should be our testing
↳data')
print(f'Training would be {total_rows - training_data}')

# the are our features and target variables (we exclude "Gender" column as it
↳serves no purpose here)
features = data[['Age', 'Smoking Status', 'Lung Capacity', 'Hospital Visits',
'Disease Type_Bronchitis', 'Disease Type_COPD', 'Disease Type_Lung
↳Cancer',
'Disease Type_Pneumonia', 'Treatment Type_Surgery', 'Treatment
↳Type_Therapy']]
target = data['Recovered']

#splits the data 80 (training) and 20 (testing)
x_train, x_test, y_train, y_test = train_test_split(features, target,
↳test_size=0.20, random_state=42, shuffle=True)

#showing the results of the split
print('\nRESULTS OF SPLIT')
print(f'Training set shape details are: x_train = {x_train.shape}, y_train=
↳{y_train.shape}')
print(f'Testing set shape details are: x_test = {x_test.shape}, y_train=
↳{y_test.shape}')

#Printing the data for the first 5 rows for both training and test datasets
print("\n--- Training Set Statistics ---")
print(x_train.head())

print("\n--- Test Set Statistics ---")
print(x_test.head())
```

We have a total of 3236 rows of data.

80% of 3236 is 2588 which should be our testing data

Training would be 648

RESULTS OF SPLIT

Training set shape details are: x_train = (2588, 10), y_train= (2588,)

Testing set shape details are: x_test = (648, 10), y_train= (648,)

--- Training Set Statistics ---

	Age	Smoking Status	Lung Capacity	Hospital Visits	\
4763	50	1	1	9	
2939	36	0	1	14	
2106	65	1	1	9	
1257	88	1	3	10	
4598	73	1	4	3	

	Disease Type_Bronchitis	Disease Type_COPD	Disease Type_Lung Cancer	\
4763	0	1	0	
2939	1	0	0	
2106	0	0	0	
1257	0	0	0	
4598	0	1	0	

	Disease Type_Pneumonia	Treatment Type_Surgery	Treatment Type_Therapy
4763	0	0	0
2939	0	1	0
2106	0	0	0
1257	1	0	1
4598	0	0	0

--- Test Set Statistics ---

	Age	Smoking Status	Lung Capacity	Hospital Visits	\
1385	22	1	1	6	
3968	84	1	1	8	
2351	89	0	4	2	
523	39	0	5	2	
4606	81	0	1	8	

	Disease Type_Bronchitis	Disease Type_COPD	Disease Type_Lung Cancer	\
1385	0	1	0	
3968	1	0	0	
2351	0	0	0	
523	0	0	0	
4606	0	0	0	

	Disease Type_Pneumonia	Treatment Type_Surgery	Treatment Type_Therapy
1385	0	0	1
3968	0	0	1
2351	1	0	1
523	1	0	1
4606	0	1	0


```

[29]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# -----
# HELPER FUNCTION: LABELS EACH BAR WITH ITS HEIGHT
# -----
def label_bars_with_values(axis):
    """
    This function goes through each bar in the given axis (axis.patches)
    and places a numeric label (the bar's height) on top of it. This is
    especially helpful for quickly reading exact values from the bar chart.
    """
    for bar in axis.patches:
        height = bar.get_height()
        if height > 0: # Only label bars with a positive height
            axis.annotate(
                f'{int(height)}', # Convert float to int for a cleaner label
                xy=(bar.get_x() + bar.get_width() / 2, height),
                xytext=(0, 5), # Offset label slightly above the bar
                textcoords='offset points',
                ha='center', # Center the label horizontally
                va='bottom', # Anchor label at the bottom
                fontsize=10,
                fontweight='bold'
            )

# -----
# 1) PRINT A USER-FRIENDLY SUMMARY OF THE TRAIN/TEST SPLIT
# -----
print("\n\033[1mSUMMARY OF TRAIN/TEST SPLIT:\033[0m\n")
print(f" • Training Features (x_train): {x_train.shape[0]} rows, {x_train.
↪shape[1]} columns")
print(f" • Testing Features (x_test) : {x_test.shape[0]} rows, {x_test.
↪shape[1]} columns")
print(f" • Training Labels (y_train): {y_train.shape[0]} rows (target only)")
print(f" • Testing Labels (y_test) : {y_test.shape[0]} rows (target only)\n")

# -----
# 2) BAR CHART: NUMBER OF ROWS IN EACH SUBSET
# -----
split_sizes = pd.DataFrame({
    'Data Subset': [
        'Training Features (x_train)',
        'Testing Features (x_test)',
        'Training Labels (y_train)',
        'Testing Labels (y_test)'
    ]
})

```

```

    ],
    'Number of Rows': [
        x_train.shape[0],
        x_test.shape[0],
        y_train.shape[0],
        y_test.shape[0]
    ]
})

# Create a figure and axis using matplotlib
plt.figure(figsize=(10, 6))
ax = plt.gca()

# Generate a "viridis" color palette with as many colors as we have subsets
colors = sns.color_palette("viridis", n_colors=len(split_sizes))

# Build the bar chart
bars = ax.bar(
    split_sizes['Data Subset'],
    split_sizes['Number of Rows'],
    color=colors,
    width=0.6
)

# Label each bar with the exact row count
labelBarsWithValues(ax)

plt.title('Number of Rows in Each Data Subset', fontsize=16, fontweight='bold')
plt.xlabel('Data Subset', fontsize=12)
plt.ylabel('Row Count', fontsize=12)

# Keep the x-axis labels straight (no rotation)
plt.xticks(rotation=0)

plt.tight_layout()
plt.show()

# Explanation for the bar chart (printed as text)
print("\033[1mHOW TO READ THE ABOVE BAR CHART:\033[0m")
print(" • Each bar represents the total number of rows (samples) in that subset.
↪")
print(" • 'Training Features' and 'Testing Features' contain multiple input
↪columns (Age, etc.).")
print(" • 'Training Labels' and 'Testing Labels' contain only the target column
↪(Recovered).")
print(" • The numbers above each bar show the exact row count for clarity.\n")

```

```

# -----
# 3) COUNT PLOT: DISTRIBUTION OF 'RECOVERED' (0 = No, 1 = Yes) IN TRAIN VS TEST
# -----
# Combine y_train and y_test into one DataFrame, labeling each row by its subset
train_df = pd.DataFrame({'Recovery Status': y_train, 'Data Subset': 'Training_
↳Set'})
test_df = pd.DataFrame({'Recovery Status': y_test, 'Data Subset': 'Testing_
↳Set'})
combined_df = pd.concat([train_df, test_df], axis=0)

plt.figure(figsize=(10, 6))
ax2 = sns.countplot(
    data=combined_df,
    x='Recovery Status',
    hue='Data Subset',
    palette='viridis'
)

# Annotate each bar with the count
for bar in ax2.patches:
    bar_height = bar.get_height()
    if bar_height > 0:
        ax2.annotate(
            f'{int(bar_height)}',
            xy=(bar.get_x() + bar.get_width() / 2, bar_height),
            xytext=(0, 5),
            textcoords='offset points',
            ha='center',
            va='bottom',
            fontsize=10,
            fontweight='bold'
        )

plt.title('Distribution of Recovery Status (0=No, 1=Yes)', fontsize=16,
↳fontweight='bold')
plt.xlabel('Recovery Status', fontsize=12)
plt.ylabel('Number of Patients', fontsize=12)
plt.legend(title='Data Subset')

plt.tight_layout()
plt.show()

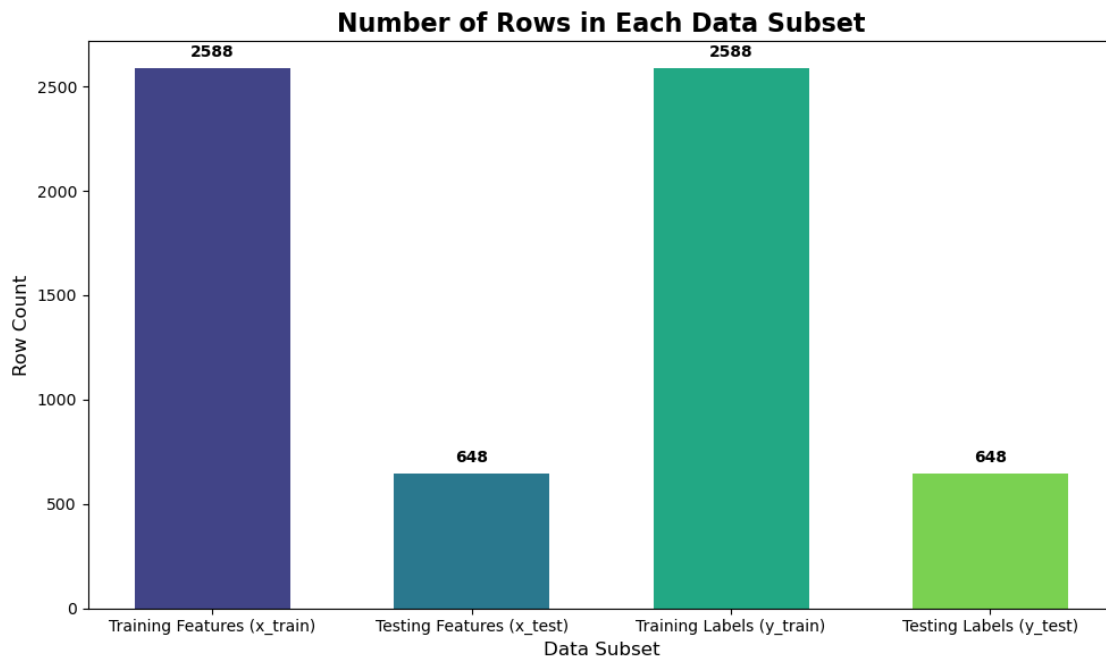
# Explanation for the count plot
print("\033[1mHOW TO READ THE ABOVE RECOVERY STATUS CHART:\033[0m")
print(" • '0' means the patient did NOT recover, and '1' means the patient_
↳recovered.")

```

```
print(" • The chart separates these counts for the Training Set vs. the Testing Set.")
print(" • Labels on top of each bar show the exact count of patients in that category.")
print(" • This comparison helps ensure both sets have a somewhat similar distribution of recovered vs. not recovered.\n")
```

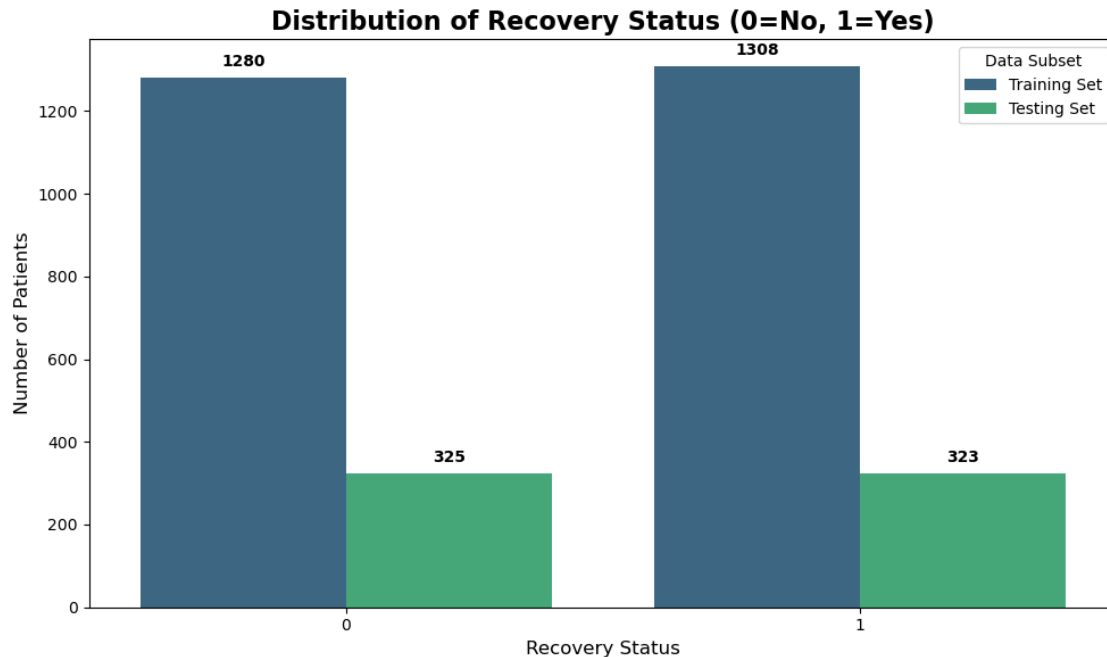
SUMMARY OF TRAIN/TEST SPLIT:

- Training Features (x_train): 2588 rows, 10 columns
- Testing Features (x_test) : 648 rows, 10 columns
- Training Labels (y_train): 2588 rows (target only)
- Testing Labels (y_test) : 648 rows (target only)



HOW TO READ THE ABOVE BAR CHART:

- Each bar represents the total number of rows (samples) in that subset.
- 'Training Features' and 'Testing Features' contain multiple input columns (Age, etc.).
- 'Training Labels' and 'Testing Labels' contain only the target column (Recovered).
- The numbers above each bar show the exact row count for clarity.



HOW TO READ THE ABOVE RECOVERY STATUS CHART:

- '0' means the patient did NOT recover, and '1' means the patient recovered.
- The chart separates these counts for the Training Set vs. the Testing Set.
- Labels on top of each bar show the exact count of patients in that category.
- This comparison helps ensure both sets have a somewhat similar distribution of recovered vs. not recovered.

```
[31]: # We calculate the mean and standard deviation for the training and test data to
      ↪ compare the model's results
train_mean = x_train.mean()
train_std = x_train.std()

test_mean = x_test.mean()
test_std = x_test.std()

# we display the mean and standard deviation to see if the results match.
print(f'The mean for the training data is {train_mean} and the standard_
      ↪ deviation is {train_std}')
print(f'The mean for the testing data is {test_mean} and the standard deviation_
      ↪ is {test_std}')
```

```
The mean for the training data is Age                    54.235317
Smoking Status              0.509274
Lung Capacity               3.004637
Hospital Visits              7.494204
```

Disease Type_Bronchitis	0.207883	
Disease Type_COPD	0.213679	
Disease Type_Lung Cancer	0.186631	
Disease Type_Pneumonia	0.194745	
Treatment Type_Surgery	0.322643	
Treatment Type_Therapy	0.334621	
dtype: float64 and the standard deviation is Age		
	20.042337	
Smoking Status	0.500011	
Lung Capacity	1.421567	
Hospital Visits	3.994532	
Disease Type_Bronchitis	0.405871	
Disease Type_COPD	0.409982	
Disease Type_Lung Cancer	0.389690	
Disease Type_Pneumonia	0.396081	
Treatment Type_Surgery	0.467578	
Treatment Type_Therapy	0.471949	
dtype: float64		
The mean for the testing data is Age		54.206790
Smoking Status	0.526235	
Lung Capacity	2.907407	
Hospital Visits	7.533951	
Disease Type_Bronchitis	0.205247	
Disease Type_COPD	0.189815	
Disease Type_Lung Cancer	0.194444	
Disease Type_Pneumonia	0.197531	
Treatment Type_Surgery	0.347222	
Treatment Type_Therapy	0.319444	
dtype: float64 and the standard deviation is Age		
	20.393512	
Smoking Status	0.499697	
Lung Capacity	1.447934	
Hospital Visits	3.940683	
Disease Type_Bronchitis	0.404194	
Disease Type_COPD	0.392457	
Disease Type_Lung Cancer	0.396078	
Disease Type_Pneumonia	0.398444	
Treatment Type_Surgery	0.476455	
Treatment Type_Therapy	0.466622	
dtype: float64		

0.1 Dataset Splitting: Rationale and Statistical Comparison

0.1.1 Rationale for Splitting

- The dataset was split into **training (80%)** and **testing (20%)** subsets.
- The train-test split is important to allow that the model to be trained on one subset of data and checked on a completely unseen subset.

- By keeping a standard 80-20 split, we ensure that there is enough data for training while preserving a portion for unbiased evaluation.
- The process of splitting the dataset also ensures that the evaluation is done on data entirely unseen, meaning that the potential risk of data leakage is decreased, ensuring reliability of its performance
- The 80-20 split also allows us to use an independent set in order to validate the performance of the model, which is an important aspect of tuning the data set

0.1.2 Statistical Comparison of Train and Test Sets

- After splitting, we compared parameters such as mean, standard deviation, and distribution of key features to ensure that the split did not introduce any significant biases.
- The describe function was applied separately to both training and test sets to analyze statistics.
- The data set can also be used to study additional advanced parameters such as median, skewness, etc in order to develop a much more thorough understanding of the data and its distribution within each subset
- We used visual tools such as plots and graphs in order to confirm that the subsets do have a similar level of variability, central tendency, and the overall spread of the data points

0.1.3 Observations

- The mean and standard deviation of key numerical variables in the training and test sets are similar, confirming that the data split is balanced.
- The proportion of categorical variables (e.g., gender, smoking status) is also maintained between both sets.
- The histogram distributions of continuous features before and after splitting show that the distribution of data remains consistent, validating the split.
- By comparing the box plots and the Interquartile Range, we can come to the conclusion that the spread and the presence of outliers in the data are consistent between both the training sets and the originals, which reinforces the need for a balanced split

0.1.4 Conclusion

Splitting the dataset correctly ensures that the model generalizes well on unseen data without overfitting. The statistical analysis after the split confirms that the training and test sets represent the original dataset proportionally. The balanced split also ensures that we have a robust model that can be used for training as well as generalization based on the consistency we notice, implying that the model will perform in the real world.

[]: